UbicacionInventario
- nombreUbicacion: String
- cantidadesPorArticulo: Map<ArticuloTienda, Integer>
+ moverArticulosA(dest: UbicacionInventario, art, cant)
+ reabastecerArticulo(art: ArticuloTienda, cant: int): void
almacena [0..*]
ArticuloTienda
- nombre: String (Raquetas, Balones, Mallas...)
- categoria: String
- precio: double
- cantidadDisponible: int
+ calcularPrecioConIVA19(): double
DetalleVenta
- cantidad: int
- precioUnitario: double
- subtotalRubro: double
+ calcularSubtotal(): double
detalle [1..*]
VentaTienda
- iva: double = 0.19 (19%)
+ calcularTotalConIVA(): double
incluye [1..*]
VentaCafeteria
- impuestoConsumo: double = 0.08 (8%)
- propinaSugerida: double = 0.10 (modificable)
- alergenosInformados: boolean
+ modificarPropina(valor: double): void
+ validarDespachoSegunInstalacion(): boolean
despacha [1..*]
«abstract»
ProductoCafeteria
# nombre: String
# precio: double
+ puedeDespacharseEn(inst: Instalacion): boolean
Bebida
- esCaliente: boolean (Fría / Caliente)
+ prohibidaEnPadelOMesaTechada(inst: Instalacion): bool
Snack
- alergenos: List<String> (Maní, Gluten, Lácteos...)
+ informarAlergenosAlSocio(): String
realizaCompra [0..*]
«abstract»
Venta
# fecha: String
# rubroContable: String (Tienda / Cafetería)
# subtotal: double
# descuentoEmpleado: double (15% propio / 8% socio)
# puntosRedimidos: int
# valorTotal: double
# puntosFidelidadGenerados: int (2% del valor)
+ aplicarCodigoDescuento(cod: String): void
+ calcularPuntosFidelidad2Porciento(): int
+ redimirPuntosComoDescuento(pts: int): void
consolida [0..*]
ReporteAdministrativo
- rubro: String (Tienda, Cafetería, Mensualidades)
- periodo: String (Diario, Semanal, Mensual)
+ generarReporteVentasPorRubroYPeriodo(): String
+ generarReporteDeportivoAsistenciaYPartidos(): String
+ generarReporteEvolucionRankings(): String
consulta [0..*]
validaQuorum [0..*]
EvaluacionDesempeno
- nota: int (Escala de 1 a 5)
- comentario: String
+ validarNota1A5(): boolean
evalua [0..1]
RegistroAsistencia
- estado: String (Asistió, Faltó, Llegó tarde)
+ agregarEvaluacionOpcional(nota: int, com: String): void
registra [1..*]
«abstract»
Usuario
# login: String
# password: String
# nombre: String
+ autenticar(pass: String): boolean
+ puedeReservarUsoPersonal(): boolean
gestiona [1..*]
Administrador
+ verEstadoInventarioInstalacionesYArticulos(): void
+ verHistorialReservasPartidosEntrenamientos(): void
+ crearODarBajaEquipoYJugador(): void
+ moverArticulosUbicacionInventario(): void
+ reabastecerTienda(): void
+ aprobarSugerenciaEmpleado(s: SugerenciaProducto): void
+ aprobarCambioTurno(sol: SolicitudCambioTurno): bool
+ puedeInscribirseAPartidoOReservarUsoPersonal(): false
aprueba [0..*]
SugerenciaProducto
- nombrePropuesto: String
- tipo: String (Artículo Tienda / Snack)
- aprobada: boolean
+ aprobarPorAdministrador(): void
propone [0..*]
radica [0..*]
SolicitudCambioTurno
- aprobada: boolean
+ mantieneMin2EntrenadoresY1Fisio(): boolean
asignadoA [0..*]
intercambia [2]
TurnoSemanal
- dia: String
- horarioAtencion: String
+ cumpleMin2EntrenadoresY1Fisio(): bool
«abstract»
Empleado
# codigoDescuento: String
# descuentoPropio: double = 0.15 (15%)
# descuentoSocios: double = 0.08 (8%)
+ obtenerDescuento(comprador: Usuario): double
+ sugerirArticuloOSnack(nom, tipo): SugerenciaProducto
+ solicitarCambioTurno(act, nvo): SolicitudCambioTurno
Fisioterapeuta
+ disponibleEnTurno(t: TurnoSemanal): bool
Entrenador
+ planearSesionEntrenamiento(): SesionEntrenamiento
+ registrarAsistencia(ses, socio, estado): void
+ registrarEvaluacionOpcional(asist, nota, com): void
+ puedeJugarConCualquierNivel(): true
planeaYDirige [0..*]
SesionEntrenamiento
- fecha: String
- hora: String
- duracion: int
+ convocarEquipo(eq: Equipo): void
+ convocarSubconjuntoSocios(socios: List<Socio>): void
ocupa [1]
«abstract»
Instalacion
# nombre: String
# capacidadJugadores: int
+ admiteModalidadYJugadores(mod, cant): boolean
+ permiteBebidaCaliente(): boolean
CanchaTenis
- capacidadSencillos: int = 2
- capacidadDobles: int = 4
+ permiteBebidaCaliente(): true
CanchaPadel
- capacidadDoblesFija: int = 4
- soloModalidadDobles: boolean = true
+ permiteBebidaCaliente(): false
MesaTenisDeMesa
- capacidadSencillos: int = 2
- capacidadDobles: int = 4
- esTechada: boolean
+ permiteBebidaCaliente(): !esTechada
ofrece [6]
ClubDeportivoSeneca
- nombre: String = 'Club Deportivo Séneca'
- minEntrenadoresPorTurno: int = 2
- minFisioterapeutasPorTurno: int = 1
+ autenticarUsuario(login, pass): Usuario
+ verificarDisponibilidadTurnos(): boolean
Tenis
- admiteSencillos: boolean = true (2 jug.)
- admiteDobles: boolean = true (4 jug.)
+ validarModalidad(mod: String, cant: int): bool
«abstract»
DisciplinaIndividual
# maxInstalacionesSimultaneas: int
# niveles: Principiante, Intermedio, Avanzado
+ nivelesIgualesOAdyacentes(n1, n2, hayCoach): bool
+ actualizarRankingELO(eloA, eloB, res): double
enDisciplina [1]
PerfilAcademiaIndividual
- nivel: String (Principiante, Intermedio, Avanzado)
- puntajeRankingELO: double
+ esIgualOAdyacente(otro: PerfilAcademiaIndividual): bool
+ actualizarPuntajeELO(nuevoELO: double): void
tieneNivelYELO [0..3]
Socio
- edad: int
- categoriaEdad: String (Sub-13, Sub-15, Sub-17, Mayores)
- puntosFidelidad: int (1 punto = 1 peso)
- tieneBebidaCaliente: boolean
+ correspondeACategoria(cat: String): boolean
+ acumularPuntos2Porciento(valorCompra: double): void
+ redimirPuntos(pts: int): boolean
+ perteneceAEquipoEnDeporte(dep: DeporteConjunto): bool
solicita [0..*]
ReservaInstalacion
- bloqueHorario: String
- cantidadJugadores: int
- modalidad: String (Sencillos / Dobles)
+ hayCupoSimultaneoEnDisciplina(): boolean
+ validarSinBebidaCaliente(s: Socio): boolean
reservaBloque [1]
«abstract»
Deporte
# nombre: String
+ esDeporteDeConjunto(): boolean
Padel
- siempreDobles: boolean = true (4 jug.)
+ validarCuatroJugadores(cant: int): bool
TenisDeMesa
- admiteSencillos: boolean = true (2 jug.)
- admiteDobles: boolean = true (4 jug.)
+ validarModalidad(mod: String, cant: int): bool
juegaEn [0..*]
InscripcionEquipo
- posicion: String
- numeroCamisetaUnico: int
+ getSocio(): Socio
+ getEquipo(): Equipo
recibeSancion [0..*]
SancionDisciplinaria
- motivo: String (2 Amarillas o 1 Roja)
- suspendidoSiguientePartido: boolean = true
+ puedeSerAlineado(): boolean
+ cumplirSuspensionSiguientePartido(): void
sanciona [0..*]
usa [1]
PartidoPracticaIndividual
- modalidad: String (Sencillos / Dobles)
- participaEntrenador: boolean
+ validarNivelesIgualesOAdyacentes(): boolean
+ actualizarPuntajeRankingELO(): void
EstadisticaFutbol
- goles: int
- asistencias: int
- tarjetasAmarillas: int
- tarjetasRojas: int
+ generaSuspensionSiguientePartido(): bool
EstadisticaBaloncesto
- puntos: int
- rebotes: int
+ getPuntos(): int, + getRebotes(): int
Futbol
- minJugadoresHabilitados: int = 7
- permiteEmpates: boolean = true
- puntosVictoria: int = 3, puntosEmpate: int = 1
- posiciones: Portero, Defensa, Medio, Delantero
+ generaSuspension(amarillas: int, rojas: int): bool
Baloncesto
- minJugadoresHabilitados: int = 5
- permiteEmpates: boolean = false
+ validarSinEmpate(ptsA, ptsB): bool
«abstract»
DeporteConjunto
# minJugadoresHabilitados: int
# permiteEmpates: boolean
# categoriasEdad: Sub-13, Sub-15, Sub-17, Mayores
+ alcanzaMinimoJugadores(habilitados: int): bool
+ otorgarPuntosLiga(victoria: bool, empate: bool): int
Voleibol
- minJugadoresHabilitados: int = 6
- permiteEmpates: boolean = false
- marcadorPorSets: boolean = true
+ validarGanadorPorSets(sA, sB): bool
deporte [1]
CompetenciaLiga (Torneo de los Cerros)
- formato: String = 'Liga todos contra todos'
- tablaPosiciones: Map<String, Integer>
+ otorgarPuntosSegunDeporte(p: PartidoCompetencia): void
+ consultarTablaPosiciones(): Map<String, Integer>
CompetenciaFases (ASCUN)
- faseActual: String (Grupos / Eliminación Directa)
+ avanzarPorPuntosYEnfrentamientoDirecto(): void
+ registrarResultadoEliminacionDirecta(): void
Equipo
- nombre: String
- categoria: String (Sub-13, Sub-15, Sub-17, Mayores)
- cupoMaximoJugadores: int
+ inscribirJugador(s: Socio, pos: String, cam: int): bool
+ camisetaDisponible(camiseta: int): boolean
+ obtenerJugadoresHabilitados(): List<Socio>
nomina [1..*]
inscribe [1..*]
«abstract»
Competencia
# nombre: String
# entidadOrganizadora: String (ASCUN / Cerros)
# anioTemporada: String
# categoriaHabilitada: String (Mayores)
# nominaMinimaHabilitados: int
+ cumpleRequisitosInscripcion(eq: Equipo): bool
+ registrarNominaOficial(jugadores: List<Socio>): void
+ obtenerGoleadorODestacado(): Socio
enfrenta [2]
PartidoInterno
- marcadorFinal: String
+ validarMismaCategoriaYDeporte(): boolean
+ validarMinimoJugadoresHabilitados(): boolean
+ aplicarSuspensionPorTarjetas(): void
programacion [1..*]
PartidoCompetencia
- institucionRival: String
- escenario: String
- marcadorFinal: String
+ validarEnNominaCompetencia(): boolean
+ verificarSinSuspensionActiva(): boolean
«abstract»
Partido
# fecha: String
# disputado: boolean
+ puedeDisputarse(): boolean
+ registrarEstadisticasIndividuales(): void
estadisticas [1..*]
«abstract»
EstadisticaJugadorPartido
+ registrarEnPartido(p: Partido, s: Socio): void
EstadisticaVoleibol
- aces: int
- bloqueos: int
+ getAces(): int, + getBloqueos(): int
EstadisticaTenisPadel
- setsGanados: int
- gamesGanados: int
- aces: int
- erroresNoForzados: int
+ getSetsGanados(): int
+ getGamesGanados(): int
EstadisticaTenisDeMesa
- setsGanados: int
- puntosTotales: int
+ getSetsGanados(): int
+ getPuntosTotales(): int